AIML331: Computer Vision Assignment 2

Keypoint-Based and Dense Tracking Across Video Frames

Due Date: 24th April (Week 7)
Total Marks: 100%
Submission Format:

- PDF report
- A compressed folder (.zip) containing your code
- Another compressed folder (.zip) containing your result images

The goal of this assignment is to explore and implement traditional feature extraction, feature matching, and tracking techniques on video frames. You will implement a sparse tracking method using SIFT keypoints and a dense tracking approach using optical flow (Lucas-Kanade). You will then analyse and compare the results.

Programming Language:
Python + Python-based OpenCV. You *may* use other languages if you really want to, but do not expect any support from the tutors or lecturers!

Data:
The data is extracted from a video and consists of individual frames saved as images in the format frame_*.jpg, where * represents the frame number in sequential order (e.g., frame_0.jpg, frame_1.jpg, frame_2.jpg, etc.).

Result Data Submission:
You will work on all the provided images, but due to submission space limitations, you only need to submit the required result images for every 5 frames as specified.

Part 1: Sparse Keypoint Detection and Tracking (40%)

Objectives:

1. Implement sparse keypoint detection and tracking using SIFT keypoints.
2. Match keypoints across consecutive frames and visualise the tracked points.

Tasks:

1. Keypoint Extraction: Extract SIFT descriptors and keypoints from each frame. **(10%)**
2. Keypoint Matching: Match the keypoints between consecutive frames using an appropriate matching technique. **(15%)**
3. Visualisation: Visualise the tracked keypoints by indicating the positions of successfully tracked points on the video frames and showing how they move from one frame to the

next. For the positions of a tracked point in two successive frames, you can draw a line connecting the two positions (previous to current) and overlay it on the current frame. **(15%)**

Deliverables:

1. Submit your code for keypoint extraction, matching, and visualisation.
2. Generate visualisation images for all frames displaying the tracked keypoints, clearly illustrating both the direction and magnitude of their motion across frames. Select every 5th frame for these result images and store them in the folder 'SIFT_Results' for submission.
3. Include an analysis in the report of the keypoint tracking method you applied to the input data. Discuss its performance under different motion patterns and speeds.

Part 2: Dense Tracking Using Optical Flow (30%)

Objectives:

1. Implement dense tracking using optical flow (Lucas-Kanade method).
2. Track pixel motion across consecutive frames and visualise the results.

Tasks:

1. Optical Flow Calculation: Compute the dense optical flow between consecutive frames using the Lucas-Kanade method. **(15%)**
2. Visualisation: Visualise the computed motion vectors as arrows or lines overlaid on the frames. Indicate the direction and magnitude of motion. **(15%)**

Deliverables:

1. Submit the code for computing optical flow and visualising the motion vectors.
2. Generate visualisation images showing the motion vectors across the frames, illustrating the flow of movement throughout all frames. Select every 5th frame for these result images and store them in the folder "OpticalFlow_Results" for submission.
3. Include an analysis in the report of the optical flow tracking method you applied to the input data. Discuss its performance under different motion patterns and speeds.

Part 3: Application on Object Tracking (30%)

Description:

You are provided with an initial bounding box with $(x, y, w, h) = (20, 353, 322, 215)$ at the first frame, where (x, y) represents the top-left pixel location, and (w, h) represent the width and height of the bounding box around the tracked car (as shown below). Your task is to analyse and compare the performance of two tracking methods (SIFT and optical flow) in

tracking specific keypoints located within the provided initial bounding box around the car object.

Tasks:

Analyse the tracking of keypoints that originate within the initial bounding box throughout the entire sequence:

1. Track the Bounding Box: Use the keypoints initially located within the provided bounding box and track them across all frames using the two separate methods: SIFT and optical flow. For each method independently, dynamically update the bounding box in each frame based on the movement of the tracked keypoints. Ensure that the bounding box stays reasonably aligned with the race car throughout the sequence, effectively following its motion over time. **(15%)**
2. Visualisation: Visualise the updated bounding box on each frame of the video, clearly showing how it moves and adapts over time to follow the race car using the tracked keypoints. The bounding box should remain reasonably aligned with the race car throughout the sequence, effectively capturing its motion without significant drift. **(15%)**

Deliverables:

1. Submit your code for tracking the bounding box.
2. Generate visualisation images to show tracked bounding box across all frames. Select every 5th frame for these result images and store them in two separate folders for submission: “BoundingBox_Results_SIFT” for the SIFT-based tracking, and “BoundingBox_Results_OpticalFlow” for the optical flow-based tracking.
3. In the report, include a comparison and analysis of the SIFT and optical flow tracking methods for object tracking, focusing on their effectiveness in following the race car throughout the video sequence.

Submission Instructions:

1. **PDF Report:**
   Submit a **PDF report** that includes:

   - Introduction: Overview of the problem and methods used (SIFT, optical flow).
   - Analysis and discussion required for each task listed above.

   Your report should be clear, concise, and well-organised, providing explanations and possible conclusions derived from the visual results or your implementation.

2. **Code Submission:**
   Submit your code in a **compressed (zip) folder**, containing:

   - Code for each part (SIFT, optical flow, bounding box tracking).
   - The code can be provided as separate files or as a single combined file. If the code is lengthy, please provide detailed guidance on how to navigate and understand it.

   Your code must be executable. Ensure it is well-documented, with clear comments explaining each key step. Additionally, provide a README file to guide us on how to review and run your code.

3. **Result Images Submission:**
   Submit your result image folders in a **compressed (zip) folder**, including: SIFT_Results, OpticalFlow_Results, BoundingBox_Results_SIFT, and BoundingBox_Results_OpticalFlow

Academic Integrity:

You must be aware and abide by VUW's academic integrity policies and regulations. Breeching these rules can lead to severe consequences, especially for repeat offences.

Plagiarism: is the copying of others' work without acknowledgement. This could include other students' assignments (in part or full) or resources online. You **must** acknowledge any content copied from anywhere except the lecture slides.

Generative AI use: you are strongly encouraged to **not** use GenAI tools (e.g. ChatGPT, Claude Code, Gemini, Copilot) when writing your code and report for this assignment. Doing this assignment the "hard way" (without GenAI) will help you to learn and practice key concepts for the closed-book exam; GenAI use has been shown to reduce cognitive activity in the brain that is key to learning. Using GenAI tools to help understand or revise **concepts** *can* be a good learning tool, however.

If you use any GenAI tools to help you do the assignment, your report **must** include an **AI Disclosure Statement**, detailing which tools you used, how you used them, and in which parts of the assignment they were used. Failing to do so will be considered a breach of academic integrity. If a large portion of the assignment is done by AI alone, your grade may be affected.